Bitácora S11: Mexi Banco

Dos bases más y una caché

Sistemas Distribuidos (Semestre 2027-1, FI-UNAM)

Victor Emiliano Calderón Gutiérrez

No. de cuenta 423097331

7 de noviembre de 2026

Entorno

Hora: 23:17 (6 de noviembre)

Para este laboratorio partí de la rama `06-poliglota-base` del repositorio https://github.com/OscarRuiz21/mexi-banco, que es la solución del lab S09 (la saga que publica sus eventos en RabbitMQ, con el outbox y el consumidor idempotente) más un Redis y un MongoDB ya instalados en el `docker-compose.yml`, pero que todavía nadie usa. La cloné dentro de mi carpeta de entregas como `s11/mexi-banco`, creé mi rama de trabajo `trabajo-s11` y, al igual que en los labs anteriores, desactivé `core.autocrlf` en el clon para que `mvnw` no se rompiera con los finales de línea de Windows. Todo lo corrí en Windows 11 con Docker Desktop y Git Bash.

Antes de levantar nada volví a cambiar el nombre del proyecto de Compose, ya que la rama base lo llama `mexi-banco-v06`, que es el mismo del lab S07, y con ese nombre iba a reusar su volumen de Postgres (donde Ana y Beto ya tienen otros saldos). Por lo que lo dejé como `mexi-banco-s11` (y la red como `mexibanco-s11`). Esta vez sí levanté todo, incluidas las tres copias de `cuenta`, ya que en la Parte 1 se trata justamente de ver que las tres comparten la caché:

```
docker compose up --build -d --wait
```

Los trece contenedores quedaron *healthy*, Redis contestó `PONG` y Mongo dio 0 avisos. Después di de alta a Ana (`...001`) con 1000, a Beto (`...002`) con 500 y a Carla (`...003`) con 300, y mandé la primera transferencia de 200 de Ana a Beto, que respondió `201` con estado `COMPLETADA`, igual que en la S09.

1. El punto de partida

Hora: 23:24

Antes de tocar el código medí cómo estaba el banco. Pedí el saldo de Ana diez veces seguidas y conté en el log de las tres copias de `cuenta` cuántas veces salió la línea `leida de Postgres`:

El conteo dio 12: las diez consultas, más dos de la transferencia que ya había hecho (la saga revisa que existan la cuenta de origen y la de destino con el mismo `GET`). Algo que también se ve es que el `leidaEn` cambia en las diez respuestas, es decir, las diez se leyeron de Postgres. Redis, por su parte, tenía sus contadores en cero, Mongo seguía vacío y el aviso de la transferencia seguía llegando a la bandeja de Postgres de la S09 (`GET /api/notificaciones?clabe=...`):

Es decir, mi "antes" es que cada consulta del saldo es una lectura a Postgres, la misma base que tiene que aguantar los cargos y los abonos.



Figura 1: Diez consultas del saldo de Ana: cada una trae un `leidaEn` distinto, y el log cuenta 12 lecturas a Postgres.

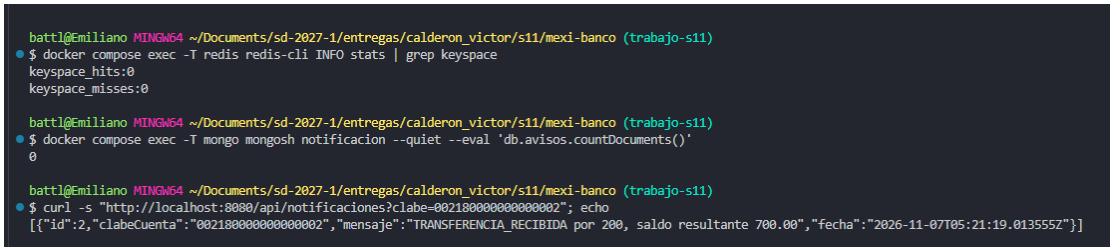


Figura 2: Redis sin aciertos ni fallos, 0 avisos en Mongo y el aviso de Beto todavía en la bandeja de Postgres.

2. La caché del saldo

Hora: 23:42

Tal como lo vimos en la clase, la idea es el patrón *cache-aside*: la consulta pregunta primero a Redis, y solo si no está va a Postgres y deja una copia para la próxima vez. En Spring no se escriben a mano los `GET`, `SET` y `DEL`, sino que los hacen dos anotaciones, por lo que el trabajo fue llenar los tres `TODO S11` de `cuenta`.

2.1 · El código

En `ConfiguracionCache` le puse `@EnableCaching` (sin ella las otras anotaciones se ignoran sin avisar) y un bean que le cambia tres cosas a la caché `cuentas` del `RedisCacheManager` que Spring Boot ya arma solo:

```
@Bean
RedisCacheManagerBuilderCustomizer cacheDeCuentas(@Value("${mexibanco.cache.ttl:30s}") Duration ttl,
                                                  ObjectMapper json, RedisConnectionFactory conexion) {
    RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
        .entryTtl(ttl)                        // 1. cada entrada vive 30 s
        .disableCachingNullValues()
        .serializeValuesWith(SerializationPair.fromSerializer(
            new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));   // 2. JSON legible
    return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
        .transactionAware()                   // 3. put y DEL esperan al commit
        .cacheWriter(...);                    // (esto es del reto, sección 7)
}
```

El TTL es lo que le pone un límite a qué tan vieja puede estar la copia (si alguien cambia el saldo por fuera de la app, como en la Parte 4, sin TTL el dato viejo se quedaría para siempre). El serializador JSON es para que el valor en Redis se pueda leer con `redis-cli` y no salga con la serialización binaria de Java, y uso el `ObjectMapper` de Spring porque ese ya sabe escribir el `Instant` de `leidaEn`. Y `transactionAware()` hace que el borrado espere al commit, lo cual explico en la pregunta 4.

Después, en `ConsultaDeSaldo.consultar`, que es la consulta que contesta `GET /cuentas/{clabe}` (lo que el cliente ve en la app), puse `@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")`. Con eso, en un *hit* contesta Redis y el método ni siquiera corre, y en un *miss* corre, lee Postgres y Spring guarda el resultado en la clave `cuentas::<clabe>`. Por último, en `CuentaService.cargar` y `abonar` puse `@CacheEvict` con la misma clave, junto al `@Transactional` que ya tenían: si el saldo cambió, la copia se borra (no se actualiza, se borra).

Lo que no toqué es `CuentaService.consultar`, ya que es la lectura que usa `cargar` para decidir si alcanza el saldo, y esa siempre tiene que ir a Postgres. Por eso la consulta que muestra vive en otra clase: además de dejar escrito en el código cuál lectura muestra y cuál decide, si se pusiera `@Cacheable` en un método llamado desde su misma clase, la llamada no pasaría por el proxy de Spring y la anotación no haría nada.

2.2 · Las tres copias comparten la caché

Reconstruí solo `cuenta` (`docker compose up -build -d cuenta`) y pedí el saldo de Ana seis veces, mostrando qué copia contestó cada vez:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ for i in 1 2 3 4 5 6; do curl -s -D - http://localhost:8080/api/cuentas/00218000000000001 | grep -iE 'x-instancia|leidaEn'; done
docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
docker compose exec -T redis redis-cli GET cuentas::00218000000000001
docker compose exec -T redis redis-cli TTL cuentas::00218000000000001
X-Instancia: ad90f7c3d1cb
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
X-Instancia: e2e137086ea7
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
X-Instancia: 0edbc309e6cf
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
X-Instancia: ad90f7c3d1cb
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
X-Instancia: e2e137086ea7
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
X-Instancia: 0edbc309e6cf
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
1
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T05:42:33.128244657Z"}
29
```

Figura 3: Seis consultas: la `X-Instancia` va cambiando entre las tres copias, pero el `leidaEn` es el mismo, Postgres se leyó una sola vez y en Redis está el JSON con 29 segundos de vida.

Como se puede observar, contestaron las tres copias de `cuenta` (`ad90f7c3d1cb`, `e2e137086ea7` y `0edbc309e6cf`), pero las seis respuestas traen el mismo `leidaEn` (05:42:33.128) y el log dice que Postgres se leyó una sola vez. Es decir, la primera consulta fue un *miss* que dejó la copia en Redis y las otras cinco fueron *hits*, aunque las contestara otra copia del servicio, ya que Redis es uno solo y lo comparten las tres (con una caché local dentro de cada copia, Postgres se habría leído tres veces). En Redis se ve el JSON de `CuentaVista` y un TTL de 29.

2.3 · La copia se borra al transferir

Para ver el orden de las operaciones dejé corriendo `redis-cli MONITOR` en otra terminal y transferí 10 de Ana a Beto. Después revisé si la copia de Ana seguía ahí y volví a pedir su saldo:

En el MONITOR se ve todo el recorrido: primero la saga revisa que existan Ana y Beto (como ninguno estaba en la caché, son dos *miss* con su `SET` de 30 segundos), después vienen los dos `DEL`, uno

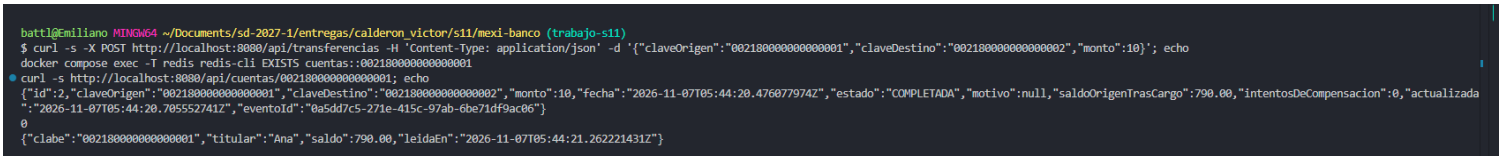


Figura 4: La transferencia de 10 queda `COMPLETADA`, `EXISTS` da 0 (la copia se borró) y la siguiente consulta ya trae 790 con un `leidaEn` nuevo.

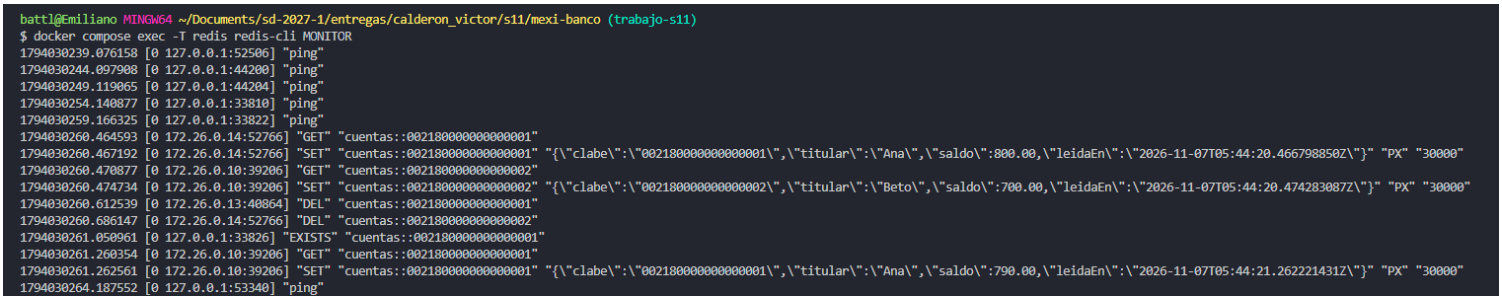


Figura 5: Lo que recibió Redis durante la transferencia: los `GET`/`SET` de las comprobaciones de la saga, los dos `DEL` después del commit del cargo y del abono, y al final el *miss* de mi consulta con su `SET ... PX 30000`.

por el cargo a Ana y otro por el abono a Beto, ya con el saldo confirmado en Postgres, después mi `EXISTS`, y al final mi consulta, que es un *miss* y vuelve a llenar la copia con el saldo nuevo de 790.

3. Mis medidas

Hora: 23:45

3.1 · Hit ratio

Para el escenario A puse en cero los contadores de Redis (`CONFIG RESETSTAT`) y pedí el saldo de Ana cien veces seguidas. Para el escenario B hice lo mismo, pero con una transferencia de 1 peso de Ana a Beto cada diez consultas:

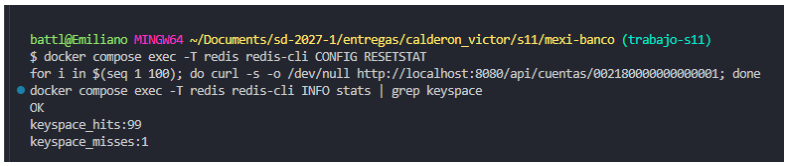


Figura 6: Escenario A: 99 aciertos y 1 fallo.

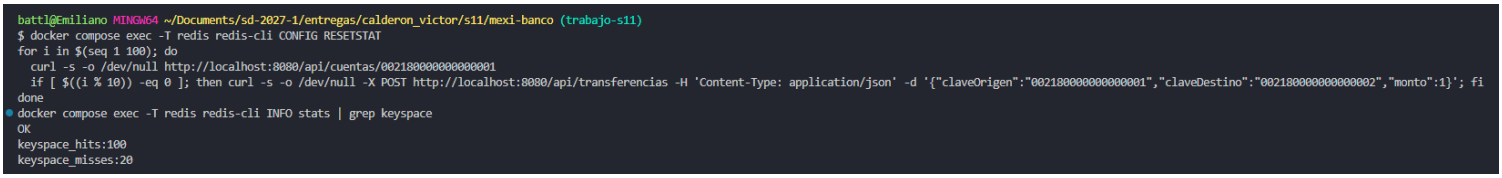


Figura 7: Escenario B: 100 aciertos y 20 fallos.

Escenario	Hits	Misses	Hit ratio
A: 100 consultas seguidas	99	1	99.0 %
B: con una transferencia cada 10	100	20	83.3 %

En A solo la primera consulta falla y las otras 99 salen de Redis (las cien caben dentro de los 30 segundos del TTL). En B baja a 83 %, porque cada transferencia borra la copia de Ana y la de Beto, por lo que hay dos fallos por transferencia: mi siguiente consulta de Ana (que ya no encuentra la copia) y la comprobación de la saga sobre Beto en la transferencia siguiente (su copia se borró en la anterior). Diez transferencias por dos fallos dan los 20. Además, en B hay más operaciones en total (120 y no 100), ya que las comprobaciones de la saga también le preguntan a Redis, y la de Ana casi siempre es un acierto.

3.2 · Latencia y lecturas a Postgres

Medí cien consultas tres veces con la caché encendida y otras tres con la caché apagada (`CACHE_TIPO=none docker compose up -d --wait cuenta`). En cada corrida conté también cuántas veces se leyó Postgres desde que empezó:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ for r in 1 2 3; do
  desde=$(date -u +%Y-%m-%dT%H:%M:%SZ)
  for i in $(seq 1 100); do curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuentas/00218000000000001; done | sort -n | awk '{t[NR]=$1*1000; s+=$1*1000} END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n",
s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
  echo "corrida $r: lecturas a Postgres = $(docker compose logs --since $desde cuenta | grep -c 'leida de Postgres')"
done
promedio=6.09 ms mediana=5.62 ms p95=7.36 ms
corrida 1: lecturas a Postgres = 1
promedio=5.57 ms mediana=5.45 ms p95=6.91 ms
corrida 2: lecturas a Postgres = 0
promedio=5.90 ms mediana=5.21 ms p95=7.07 ms
corrida 3: lecturas a Postgres = 0
```

Figura 8: Con caché: tres corridas de 100 consultas, con 1, 0 y 0 lecturas a Postgres.

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ CACHE_TIPO=none docker compose up -d --wait cuenta
 ✓ Container mexi-banco-s11-discovery-1 Healthy                                                
```

Una aclaración sobre las lecturas sin caché: en la captura salen 100, 107 y 118, pero en realidad son 100 en cada corrida. Lo que pasa es que el `-since` de `docker compose logs` lo calculé con `date` al segundo, por lo que cada corrida también cuenta las últimas lecturas de la anterior que cayeron en ese mismo segundo. Con caché eso no se nota, porque casi no hay lecturas.

Como se puede ver, la diferencia de latencia es muy chica: con caché la mediana anda entre 5.2 y 5.6 ms y sin ella entre 5.8 y 6.2 ms, es decir, menos de un milisegundo. Esto tiene sentido en mi laptop, ya que Postgres tiene la tabla (de tres filas) en memoria y está a un salto de red, igual que Redis, y la mayor parte de esos 5 ms se van en el gateway y en el balanceo. Lo que sí cambia muchísimo es la otra columna: con caché, 300 consultas le costaron a Postgres una sola lectura; sin caché, le costaron 300.

4. Los avisos en MongoDB

Hora: 00:19 (7 de noviembre)

En `notificacion` los avisos de las transferencias se mudan de la bandeja de Postgres a la colección `avisos` de MongoDB, un documento por aviso. Todo está en `avisos/BitacoraDeAvisos`, donde llené los cuatro métodos con `TODO S11`.

4.1 · El código

El método `documento(...)` arma el aviso como un `org.bson.Document`: primero los campos comunes del contrato y luego un `switch` sobre el `Contacto` que agrega los del canal. Como `Contacto` es una interfaz sellada, si al `switch` le faltara un canal no compilaría, que es justo lo que queremos (si mañana alguien agrega WhatsApp, se rompe en compilación y no en producción):

```
Document aviso = new Document("_id", evento.eventoId() + ":" + destinatario.clabe())
    .append("eventoId", evento.eventoId().toString())
    .append("transferenciaId", evento.transferenciaId())
    .append("clabe", destinatario.clabe())
    .append("tipo", destinatario.tipo())
    .append("monto", new Decimal128(evento.monto()))      // dinero: Decimal128, nunca double
    .append("saldo", new Decimal128(destinatario.saldo()))
    .append("fecha", Date.from(evento.fecha()));          // la fecha del evento, no la de hoy
switch (contacto) {
    case Contacto.Sms sms -> aviso.append("canal", "SMS")
        .append("telefono", sms.telefono()).append("texto", texto);
    case Contacto.Correo correo -> aviso.append("canal", "CORREO")
        .append("correo", correo.correo()).append("asunto", asunto(destinatario.tipo())).append("cuerpo
    ", texto);
    case Contacto.Push push -> aviso.append("canal", "PUSH")
        .append("token", push.token()).append("titulo", asunto(destinatario.tipo())).append("cuerpo",
    texto);
}
```

El `_id` lo pone el banco y no Mongo, y es `eventoId:clabe`, ya que una `TransferenciaRealizada` genera dos avisos (uno para quien envía y otro para quien recibe), y el mismo evento con la misma CLABE siempre da el mismo `_id`. Para el asunto del correo y el título del push hice un método `asunto(tipo)` con otro `switch` (“Enviaste una transferencia”, “Recibiste una transferencia” o “Tu transferencia no se realizo”).

Después, `registrar(evento)` inserta los avisos uno por uno y regresa cuántos eran nuevos; si Mongo contesta `DuplicateKeyException`, ese aviso ya estaba, por lo que no es un error, no se cuenta y se sigue con el siguiente. Los inserto uno por uno y no todos juntos porque un `insert` de varios se detiene en el primer `_id` repetido, y si el proceso se hubiera caído después de guardar el primer aviso, la reentrega tiene que poder guardar el segundo:

```
for (Destinatario destinatario : destinatarios(evento)) {
    Document aviso = documento(evento, destinatario, contactos.de(destinatario.clabe()));
    try {
        mongo.insert(aviso, COLECCION);
        nuevos++;
    } catch (DuplicateKeyException yaEstaba) {
        log.info("[avisos] {} ya estaba: evento repetido, no se duplica", aviso.get("_id"));
    }
}
```

Por último, `crearIndice()` crea al arrancar el índice `clabe_fecha` (`{clabe: 1, fecha: -1}`), que sigue la forma de la consulta de `ultimos(clabe)`: una `Query` con `Criteria.where("clabe").is(clabe)`, ordenada por `fecha` descendente y con `limit(10)`. Y en el consumidor de RabbitMQ de la S09 cambié la línea que guardaba en la bandeja de Postgres por `nuevos = bitacora.registrar(evento)`. Lo cambié y no agregué, ya que guardar en las dos bases sería una doble escritura sin una transacción común (justo el problema de la S09): los avisos se mudan, no se copian.

4.2 · Las tres formas

Reconstruí `notificacion` y en su log apareció el índice:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-07T05:50:53.147Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos            : [avisos] indice clabe_fecha listo en avisos
```

Figura 10: Al arrancar, `notificacion` crea el índice `clabe_fecha` en la colección `avisos`.

Luego mandé 50 de Ana a Beto (la transferencia 13) y 50 de Ana a Carla (la 14), y busqué en Mongo los avisos de cada una:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval 'db.avisos.find({transferenciaId: 13}).sort({fecha: 1})'
[
  {
    _id: 'b08fa5f0-efe6-4b86-9e2d-aff6bb59c3d1:00218000000000001',
    eventoId: 'b08fa5f0-efe6-4b86-9e2d-aff6bb59c3d1',
    transferenciaId: Long('13'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('730.00'),
    fecha: ISODate('2026-11-07T06:19:01.595Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 730.00'
  },
  {
    _id: 'b08fa5f0-efe6-4b86-9e2d-aff6bb59c3d1:00218000000000002',
    eventoId: 'b08fa5f0-efe6-4b86-9e2d-aff6bb59c3d1',
    transferenciaId: Long('13'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50'),
    saldo: Decimal128('770.00'),
    fecha: ISODate('2026-11-07T06:19:01.595Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 770.00'
  }
]
```

Figura 11: Los avisos de la transferencia 13: un SMS para Ana (con `telefono` y `texto`) y un CORREO para Beto (con `correo`, `asunto` y `cuerpo`).

Como se puede observar, los cuatro documentos tienen los mismos campos de arriba (`_id`, `eventoId`, `transferenciaId`, `clabe`, `tipo`, `monto` y `saldo` como `Decimal128`, `fecha` como `ISODate` y `canal`), pero lo de abajo depende del canal, y nadie tuvo que hacer un `ALTER TABLE` para que eso existiera. Además,

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval 'db.avisos.find({transferenciaId: 14}).sort({fecha: 1})'
[
  {
    _id: '0cadac43-27a0-483d-9dff-ec9c4da0b5aa:00218000000000001',
    eventoId: '0cadac43-27a0-483d-9dff-ec9c4da0b5aa',
    transferenciaId: Long('14'),
    clabe: '00218000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('680.00'),
    fecha: ISODate('2026-11-07T06:19:01.879Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 680.00'
  },
  {
    _id: '0cadac43-27a0-483d-9dff-ec9c4da0b5aa:00218000000000003',
    eventoId: '0cadac43-27a0-483d-9dff-ec9c4da0b5aa',
    transferenciaId: Long('14'),
    clabe: '00218000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50'),
    saldo: Decimal128('350.00'),
    fecha: ISODate('2026-11-07T06:19:01.879Z'),
    canal: 'PUSH',
    token: 'fcm:demo-00218000000000003',
    titulo: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00'
  }
]
```

Figura 12: Los avisos de la transferencia 14: otro SMS para Ana y un PUSH para Carla (con `token`, `titulo` y `cuerpo`).

el `_id` de cada aviso es el `eventoId` seguido de la CLABE de quien lo recibe. El esquema, en cambio, lo aplica quien lee: si pido solo los avisos que tienen teléfono, Mongo me regresa los dos de Ana:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --quiet --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: 'b08fa5f0-efe6-4b86-9e2d-aff6bb59c3d1:00218000000000001',
    clabe: '00218000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '0cadac43-27a0-483d-9dff-ec9c4da0b5aa:00218000000000001',
    clabe: '00218000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

Figura 13: Solo los avisos que tienen `telefono`: los dos SMS de Ana.

4.3 · El endpoint con los últimos 10

Para comprobar el límite hice doce transferencias de 1 peso de Ana a Beto y pedí la bitácora de Beto por el gateway. Para que se pudiera leer, formateé la respuesta y dejé un renglón por aviso con sus campos principales, y al final conté cuántos avisos regresó:

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s11/mexi-banco (trabajo-s11)
$ for i in $(seq 1 12); do curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":1}'; done
curl -s http://localhost:8080/api/notificaciones/00218000000000002 | python -m json.tool | grep -E '"(transferenciaId|canal|monto|fecha)"' | paste - - - -
curl -s http://localhost:8080/api/notificaciones/00218000000000002 | grep -o '"canal"' | wc -l
        "transferenciaId": 25,          "monto": 1,             "fecha": "2026-11-07T06:20:09.666+00:00",               "canal": "CORREO",
        "transferenciaId": 24,          "monto": 1,             "fecha": "2026-11-07T06:20:09.583+00:00",               "canal": "CORREO",
        "transferenciaId": 23,          "monto": 1,             "fecha": "2026-11-07T06:20:09.497+00:00",               "canal": "CORREO",
        "transferenciaId": 22,          "monto": 1,             "fecha": "2026-11-07T06:20:09.412+00:00",               "canal": "CORREO",
        "transferenciaId": 21,          "monto": 1,             "fecha": "2026-11-07T06:20:09.317+00:00",               "canal": "CORREO",
        "transferenciaId": 20,          "monto": 1,             "fecha": "2026-11-07T06:20:09.227+00:00",               "canal": "CORREO",
        "transferenciaId": 19,          "monto": 1,             "fecha": "2026-11-07T06:20:09.133+00:00",               "canal": "CORREO",
        "transferenciaId": 18,          "monto": 1,             "fecha": "2026-11-07T06:20:09.046+00:00",               "canal": "CORREO",
        "transferenciaId": 17,          "monto": 1,             "fecha": "2026-11-07T06:20:08.959+00:00",               "canal": "CORREO",
        "transferenciaId": 16,          "monto": 1,             "fecha": "2026-11-07T06:20:08.870+00:00",               "canal": "CORREO",
10
```

Figura 14: `GET /api/notificaciones/00218000000000002`: exactamente 10 avisos, todos CORREO de 1 peso, de la transferencia 25 a la 16, del más reciente al más viejo.

Es decir, de las doce transferencias el endpoint solo regresa las últimas diez (de la 25 a la 16), con la fecha bajando, que es justo lo que pide el contrato.

4.4 · El índice trabaja

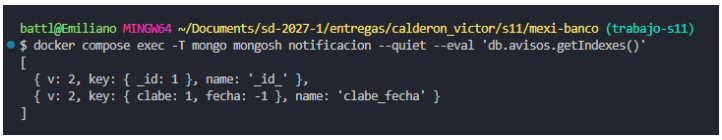


Figura 15: Los índices de la colección: el de `_id` y `clabe_fecha`.

Después le pedí a Mongo el plan de la consulta de `ultimos()` con `explain`, primero normal y luego obligándolo a no usar el índice con `hint({$natural: 1})`. Como la salida completa de `explain` es muy larga, imprimí solo las etapas y los contadores:

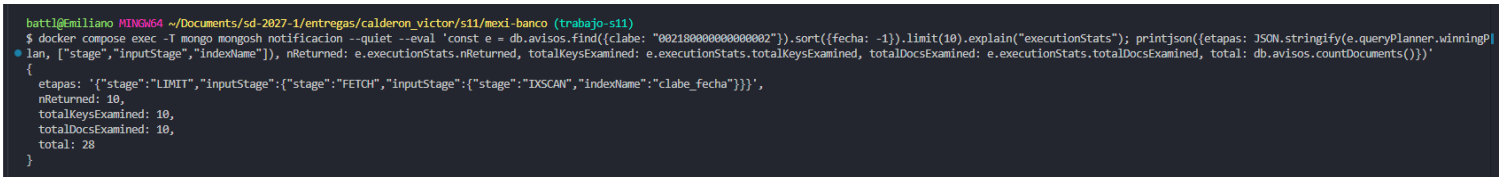


Figura 16: Con índice: `LIMIT` ← `FETCH` ← `IXSCAN` sobre `clabe_fecha`, 10 llaves y 10 documentos examinados.

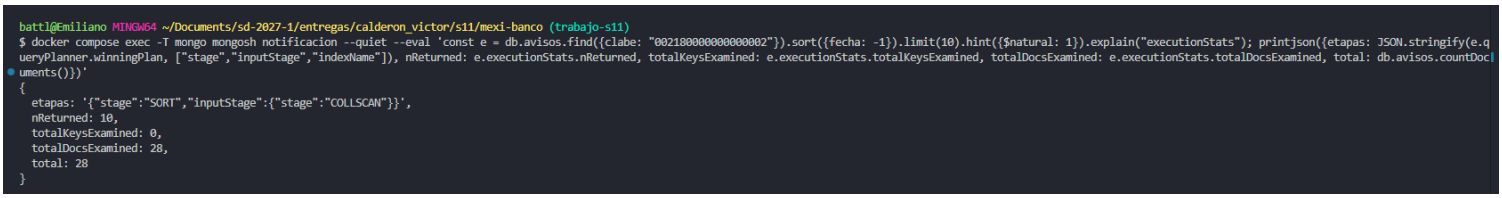


Figura 17: Sin índice: `SORT` ← `COLLSCAN`, 28 documentos examinados (todos) para regresar 10.

	Etapas	Documentos examinados	Regresados
Con índice	LIMIT ← FETCH ← IXSCAN	10	10
Sin índice	SORT ← COLLSCAN	28 (todos)	10

Con el índice, Mongo va directo a las entradas de Beto, que ya están ordenadas por fecha, y se detiene en la décima; sin él, recorre la colección completa y ordena en memoria. Con 28 avisos la diferencia es chica, pero la consulta sin índice crece con toda la colección, mientras que con índice siempre lee 10.

5. El duplicado y el dato viejo

Hora: 00:27

5.1 · El evento duplicado

La mensajería de la S09 entrega "al menos una vez", por lo que un evento puede llegar dos veces. Lo provoqué publicando el mismo evento (el `11111111-...`, con CLABEs inventadas que terminan en 091 y 092) dos veces desde la API del panel de RabbitMQ, y luego conté sus avisos en Mongo:

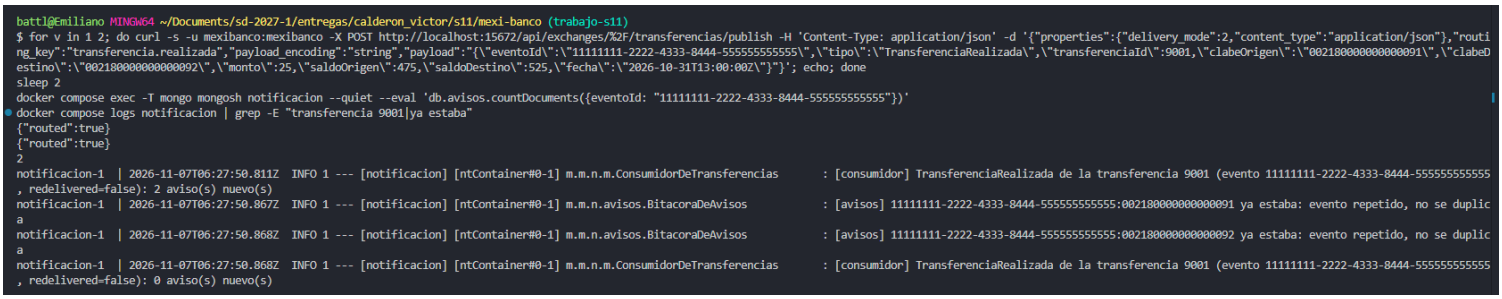


Figura 18: El mismo evento publicado dos veces: los dos llegan (`routed:true`), pero en Mongo quedan 2 avisos y no 4. La primera entrega guardó 2 nuevos y la segunda 0, con sus dos "ya estaba".

Quedaron dos documentos y no cuatro. La primera entrega guardó los dos avisos (el de la 091 y el de la 092), y la segunda armó exactamente los mismos dos `_id`, por lo que Mongo rechazó los dos `insert`, `registrar` los contó como "ya estaba" y el consumidor reportó 0 avisos nuevos (y le hizo `ack` al mensaje de todos modos, ya que no es un error). Es decir, el `_id` hace de llave única y nos da la misma idempotencia que en la S09 tenía la tabla `evento_procesado`, pero sin tabla aparte.

5.2 · El dato viejo

Aquí la idea es ver que la caché puede mentir. Primero guardé el saldo real de Ana en una variable (668), la consulté para que su saldo quedara en la caché, y luego cambié su saldo directo en Postgres a 5, como lo haría un script mal hecho, es decir, sin pasar por `cuenta` y por lo tanto sin `@CacheEvict`. Después la consulté cada 5 segundos, mostrando el TTL que le quedaba a la copia:



Figura 19: Después del `UPDATE`, el cliente sigue viendo 668 con el mismo `leidaEn` mientras baja el TTL; cuando la copia caduca (TTL -2) se lee Postgres y aparece el 5 con un `leidaEn` nuevo.

Mientras el saldo viejo seguía en la caché, en otra terminal intenté transferir 100 de Ana a Beto:

El cliente vio un saldo que ya no existía durante unos 32 segundos: la copia se había leído a las 06:28:41.67 (eso dice su `leidaEn`), el `UPDATE` fue justo después, y la primera respuesta con el saldo real trae `leidaEn` 06:29:14.79. Es decir, `leidaEn` me dice la edad del dato que estoy viendo sin abrir Redis, y lo más que puede atrasarse es el TTL (más lo que tarde en llegar la siguiente consulta).

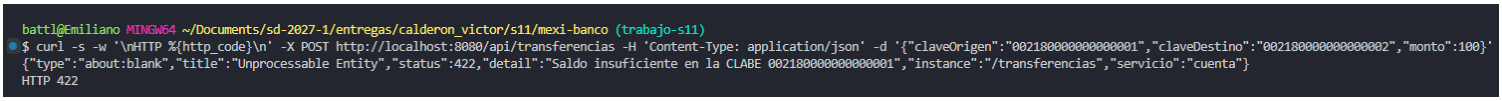


Figura 20: Con la app mostrando 668, la transferencia de 100 responde `422`: "Saldo insuficiente".

Sin embargo, la transferencia no se dejó engañar: respondió `422` "Saldo insuficiente en la CLABE 002180000000000001", porque el cargo autoriza con `CuentaService.consultar`, que lee Postgres, y ahí había 5. La caché engañó a la pantalla, pero no a la decisión. Al final regresé el saldo de Ana a 668 con el mismo `UPDATE` y borré su copia de Redis.

6. Preguntas de cierre

1. ¿Por qué MongoDB para los avisos y no para el saldo? Porque los avisos son un caso de documentos: cada uno tiene una forma distinta según su canal (el SMS lleva teléfono, el correo dirección y asunto, el push el token), se escriben una sola vez, no se actualizan y se leen enteros por CLABE, sin joins ni reglas entre un aviso y otro. En una tabla eso sería una columna por cada campo de cualquier canal (casi todas vacías) o una tabla por canal, y en Mongo cada documento trae lo suyo, como se vio en la sección 4.2. El saldo es lo contrario: siempre tiene la misma forma, se actualiza todo el tiempo y es dinero, por lo que necesita que un cargo y su asiento se confirmen juntos o no se confirmen, que dos cargos al mismo tiempo no se pisen (el `@Version` de `Cuenta`) y que nunca se pierda. Esa garantía ya la tenemos en Postgres con una transacción local, y no hay nada que ganar moviéndola a otro motor.
2. ¿Qué hit ratio medí y qué compré con la caché? En A medí 99 % (99 aciertos y 1 fallo) y en B 83 % (100 y 20). Cambia porque cada transferencia borra la copia de Ana y la de Beto, por lo que la siguiente lectura de cada uno falla: dos fallos por transferencia, diez transferencias, veinte fallos. Mientras más seguido cambie un saldo, menos sirve su copia. Con mis números, la latencia casi no cambió (mediana de 5.2–5.6 ms con caché contra 5.8–6.2 ms sin ella), pero las lecturas a Postgres pasaron de 300 a 1 en las mismas 300 consultas. Es decir, lo que compré fue capacidad más que velocidad: Postgres queda libre para los cargos y abonos, que son los que no pueden salir de la caché. En mi laptop, con una tabla de tres filas, Postgres contesta casi igual de rápido que Redis; con millones de clientes abriendo la app, la diferencia estaría en que Postgres no se satura.
3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché? En la Parte 4 la caché decía 668 mientras Postgres decía 5; si el cargo hubiera autorizado con la caché, la transferencia de 100 se habría aprobado con dinero que ya no existía, y Ana habría quedado en −95 (o el `UPDATE` del cargo habría restado sobre el 5 real). Y no hace falta un script mal hecho: basta con dos cargos seguidos dentro de la ventana en la que la copia todavía no se ha borrado. Por eso la regla de "la caché muestra, nunca decide": mostrar un saldo de hace 30 segundos es una molestia, pero autorizar con él es perder dinero.
4. ¿Por qué `DEL` y no `SET` con el saldo nuevo? ¿Y por qué después del commit? Se borra porque un `SET` podría dejar en Redis un valor que no es el último: si dos movimientos de la misma cuenta terminan casi al mismo tiempo en dos copias distintas de `cuenta`, sus `SET` pueden llegar a Redis en el orden contrario y el saldo viejo pisaría al nuevo hasta que caduque. Con `DEL` no importa el orden: la siguiente consulta lee Postgres, que es la verdad. Además, solo se paga la lectura cuando alguien de verdad pide el saldo. Y va después del commit porque si borrara antes, en la ventana entre el `DEL` y el commit otra consulta podría hacer un *miss*, leer el saldo viejo (que todavía no se confirma) y volver a dejarlo en la caché, ahora por 30 segundos. Además, si la transacción hiciera rollback, se habría borrado por nada. Eso es lo que hace `transactionAware()`, y en el MONITOR de la sección 2.3 se ve el `DEL` justo después de cada movimiento.

7. Reto: Redis se cae

Hora: 00:32

Primero probé qué pasaba con mi versión de la Parte 1 si apagaba Redis (`docker compose stop redis`):



Figura 21: Antes del arreglo: con Redis apagado, el GET del saldo y la transferencia responden `500`, y el log dice `RedisCommandTimeoutException`.

Tanto el GET como la transferencia respondieron `500` en medio segundo (el `timeout` de 500 ms de `spring.data.redis`), es decir, una copia que solo servía para mostrar estaba tumbando las consultas y también las transferencias, ya que la saga revisa las cuentas con ese mismo GET. En este caso la transferencia falló antes del cargo y los saldos no se movieron, pero el problema de fondo es peor, y se ve al pensar dónde atrapar el error.

7.1 · Por qué no alcanza un `CacheErrorHandler`

Spring trae `CacheErrorHandler`, que atrapa los errores de la caché, y parece la respuesta. Sin embargo, con `transactionAware()` el `@CacheEvict` no borra en el momento: solo registra el `DEL` para después del commit. El `CacheErrorHandler` envuelve esa llamada (que no falla), pero no el `DEL` real, que corre después, en el `afterCommit` de la transacción y fuera de su alcance. Si ese `DEL` truena, el cargo ya está confirmado en Postgres pero `cuenta` contesta 500, y la saga cree que el cargo no ocurrió.

Por eso el arreglo lo puse una capa más abajo, en el `RedisCacheWriter`, que es la pieza que de verdad habla con Redis (todas las operaciones pasan por ahí, incluido el `DEL` de después del commit). Hice una clase `CacheWriterTolerante` que envuelve al writer de Spring y, si Redis falla, lo deja en el log y sigue como si la caché estuviera vacía: un `GET` es un *miss* (se lee Postgres) y un `SET` o un `DEL` no hacen nada. Se conecta con `.cacheWriter(...)` en el builder de la sección 2.1:

```
.cacheWriter(new CacheWriterTolerante(RedisCacheWriter.nonLockingRedisCacheWriter(conexion)));
```

Al probar esa primera versión me encontré con otro problema: el GET ya respondía 200, pero tardaba 5 segundos, y una transferencia de prueba de 1 peso de Carla a Beto (la 28) terminó en `COMPENSADA`, aunque el abono a Beto sí se había aplicado. Investigando, vi que con el contenedor de Redis apagado, cada intento de reconexión resuelve el nombre `redis` por DNS y eso tarda unos 5 segundos (lo medí con `getent hosts redis` dentro de `cuenta`: 5.2 s). Eso es más que los 2 segundos con los que la saga espera a `cuenta`, por lo que la saga dio por fallido un abono que sí ocurrió y le devolvió el dinero a Carla. Es decir, justo el riesgo del que avisaba la guía: que `cuenta` conteste mal (o tarde) sobre una operación que ya se confirmó. (Por esa prueba Beto quedó con 1 peso de más, que aparece en los saldos de las capturas siguientes.)

Entonces le agregué dos cosas al writer:

```
static final Duration LIMITE = Duration.ofMillis(500);    // lo mas que se espera a Redis
static final Duration PAUSA = Duration.ofSeconds(10);     // sin Redis despues de una falla

private <T> T intentar(String operacion, String nombre, byte[] clave, Supplier<T> accion, T siFalla) {
    if (System.nanoTime() - sinRedisHasta.get() < 0) {
        return siFalla;                                    // en pausa: ni lo intento
    }
    try {
        return CompletableFuture.supplyAsync(accion, HILOS).get(LIMITE.toMillis(), MILLISECONDS);
    } catch (TimeoutException tardoDeMas) {
        fallo(operacion, nombre, clave, ...);              // abre la pausa y lo dice en el log
    } catch (ExecutionException error) { ... }             // lo mismo si Redis contesta con error
    return siFalla;
}
```

Cada operación con Redis tiene un tope de 500 ms (el mismo `timeout` de `spring.data.redis`), y si no contesta en ese tiempo cuenta como falla, aunque el DNS siga atorado. Y después de una falla, durante 10 segundos esa copia de `cuenta` ni siquiera intenta Redis y va directo a Postgres, por lo que solo la primera operación paga la espera. Pasada la pausa lo vuelve a intentar sola. El riesgo que acepto es que si un `DEL` falla (o se lo salta la pausa), una copia vieja podría quedarse en Redis hasta su TTL, igual que en la Parte 4: se mostraría un saldo de hasta 30 segundos atrás, pero nunca se decide con él. (Y si Redis se reinicia, arranca vacío, porque no tiene persistencia.)

7.2 · Con el arreglo

Reconstruí `cuenta`, apagué Redis otra vez y repetí la prueba con Ana y Beto:



Figura 22: Con Redis apagado: los GET responden 200 (las tres primeras, una por copia de `cuenta`, en medio segundo; la cuarta en 8 ms), la transferencia de 5 responde `201 COMPLETADA` en 0.11 s y los saldos cuadran: Ana de 668 a 663 y Beto de 784 a 789.

Cada copia de `cuenta` pagó una vez los 500 ms del tope (y lo dejó en el log), y a partir de ahí contestó desde Postgres sin esperar: la cuarta consulta, que volvió a caer en la primera copia, tardó 8 ms, y la transferencia completa tardó 0.11 segundos, ya que sus comprobaciones, el cargo y el abono cayeron dentro de la pausa. Lo que el cliente pierde es solo la copia: el saldo se lee de Postgres en cada consulta, como en la Parte 0.

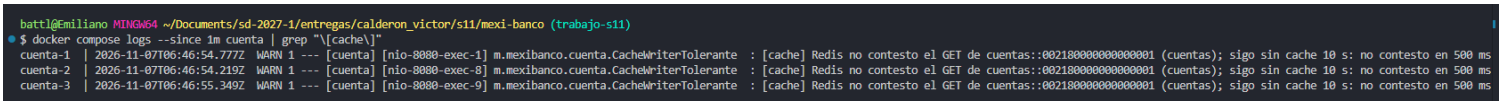


Figura 23: El log dice qué operación de caché falló: el `GET` de `cuentas::00218000000000001` en cada copia, porque Redis no contestó en 500 ms.

Al volver a encender Redis (`docker compose start redis`) noté que la caché no regresa en el instante: el cliente de Redis que usa Spring (Lettuce) reintenta la conexión con esperas cada vez más largas, y como Redis estuvo apagado casi dos minutos, ya iba en intentos cada 35 segundos. En cuanto se reconectó, todo siguió solo:

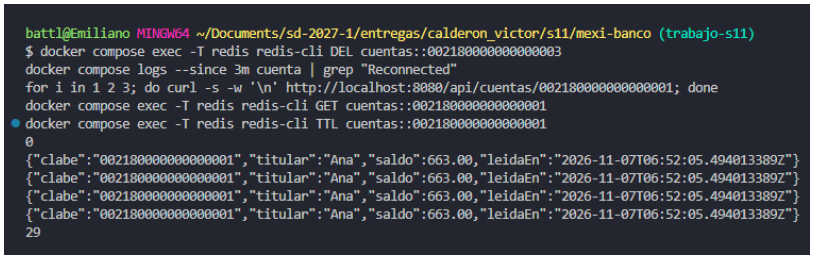


Figura 24: Con Redis de vuelta: las tres consultas traen el mismo `leidaEn` y en Redis está otra vez la copia de Ana con su TTL.

Es decir, con Redis apagado el GET responde 200, la transferencia responde 201, los saldos cuadran y el log dice qué operación de caché falló, y al encender Redis la caché vuelve a usarse sin reiniciar nada. La caché era solo una copia para mostrar y ahora se comporta como tal: si no está, el banco es un poco más lento, pero no se cae.

Entrega

En mi carpeta `entregas/calderon_victor/s11/` quedan esta bitácora y la carpeta `codigo/`, con los dos servicios completos (sin `target/`) y el `docker-compose.yml`, que sí cambié:

- `codigo/cuenta/`: la caché del saldo (`ConfiguracionCache`, `@Cacheable` en `ConsultaDeSaldo` y `@CacheEvict` en `cargar` y `abonar`) y, del reto, `CacheWriterTolerante`.
- `codigo/notificacion/`: `BitacoraDeAvisos` con los avisos en MongoDB, el índice y los últimos 10, y el consumidor guardando solo en Mongo.
- `codigo/docker-compose.yml`: solo el nombre del proyecto y de la red (`mexi-banco-s11`).

Corrí las pruebas que ya traían los dos servicios con `./mvnw test` (con la misma opción de la S09, ya que tengo Java 25 y el Mockito de Spring Boot 3.3.4 todavía no lo reconoce):

```
./mvnw test -DargLine=-Dnet.bytebuddy.experimental=true
```

En `cuenta` pasan las 10. En `notificacion` pasan 11 de 16: las 5 que fallan son de `ConsumidorDeTransferenciasTest`, que todavía revisa que el consumidor guarde en la bandeja de Postgres. La guía avisa que eso es esperado después de la Parte 2 (y Docker no corre las pruebas, por lo que no afecta a la imagen), así que no las modifiqué.